

Name : Neil Taison RIGAUD

Student ID : 411221367

Subject : Intro to Image Processing – Assignment #1

Date : 2025/09/18

1- Relationship between resolution and file color/gray images for an image resolution of 512 * 512 with *four* number after the decimal point:

A *color image* consists of 3 channels in case of RGB:

$$512 * 512 * 3 = 786432 \text{ bytes}$$

Converting to Kb:

$$786432 \text{ bytes} = 786.4320 \text{ Kb}$$

Converting to Mb:

$$786432 \text{ bytes} = 0.7864 \text{ Mb}$$

A *grayscale image* only has 1 channel:

$$512 * 512 * 1 = 262144 \text{ bytes}$$

Converting to Mb:

$$262144 \text{ bytes} = 0.2621 \text{ Mb}$$

2- Transformation of color image into gray image, and display of color/gray images using:

$$\text{grayscale image} = R * 0.299 + G * 0.587 + B * 0.114$$

Program procedure listing:

- a) Import standard C++ libraries and I/O libraries for **loading** and **writing** image data **only**

```
1 #define STB_IMAGE_IMPLEMENTATION
2 #define STB_IMAGE_WRITE_IMPLEMENTATION
3
4 #include "stb_image.h"
5 #include "stb_image_write.h"
6
7 #include <cstdlib>
8 #include <iostream>
9 #include <string>
```

- We use the header files from the [stb single-file public domain libraries for C/C++](#):
 - o **stb_image_h** to load and decode the image file
 - o **stb_image_write** to write the converted grayscale image to disk

- b) Define constant RGB weight values from the formula and I/O filenames:

```

11 constexpr double R_WEIGHT = 0.299;
12 constexpr double G_WEIGHT = 0.587;
13 constexpr double B_WEIGHT = 0.114;
14
15 int main() {
16     try {
17         std::string input_png = "lena.png";
18         std::string output_png = "lena_gray.png";

```

c) Load the image data using the **stbi_load** function:

```

20     int width = 0;
21     int height = 0;
22     int channels = 0;
23     unsigned char *img =
24         stbi_load(input_png.c_str(), &width, &height, &channels, 3);
25     if (!static_cast<bool>(img)) {
26         throw std::runtime_error("Failed to load image: " + input_png);
27     }

```

- **stbi_load** receives five (5) parameters:
 - Filename
 - Pointers to memory address to store:
 - Picture width
 - Picture height
 - Number of channels in file
 - Number of desired channels
 - In our case 3 for RGB

d) Conversion to grayscale *in-place*

```

29     // Convert to grayscale in-place
30     for (int i = 0; i < width * height * 3; i += 3) {
31         unsigned char r = img[i];
32         unsigned char g = img[i + 1];
33         unsigned char b = img[i + 2];
34         auto gray = static_cast<unsigned char>((R_WEIGHT * r) + (G_WEIGHT * g) +
35                                                (B_WEIGHT * b));
36         img[i] = img[i + 1] = img[i + 2] = gray;
37     }

```

- Apply the given *grayscale conversion formula* to each of the pixel data collected in the previous array
 - Collect the pixel data for each channel
 - Compute the **grayscale value** with the RGB pixel data
 - Re-assign each pixel values for the three RGB channels to the obtained **grayscale value**

e) Write the converted grayscale image data to the output file using **stbi_write_png**:

```

39     if (!static_cast<bool>(stbi_write_png(output_png.c_str(), width, height, 3,
40                                         img, width * 3))) {
41         stbi_image_free(img);
42         throw std::runtime_error("Failed to save image: " + output_png);
43     }
44
45     stbi_image_free(img);
46     std::cout << "Grayscale PNG saved to " << output_png << "\n";
47

```

- **stbi_write_png** receives five (5) parameters:
 - o The output file name
 - o Width of the image in pixels
 - o Height of the image in pixels
 - o Number of color channels per pixel in data (3)
 - o Pointer to raw pixel data from top-row
 - o **strides_in_bytes**: number of bytes between the start of one row and the start of the next
- We also use **stbi_image_free** to free the pixel buffer returned by **stbi_load** when we were loading the image file.

f) Open the output file optionally

```

48     // Optional: open the file
49     std::string cmd;
50     #if defined(_WIN32)
51         cmd = "start " + output_png;
52     #elif defined(__APPLE__)
53         cmd = "open " + output_png;
54     #elif defined(__linux__)
55         cmd = "xdg-open " + output_png;
56     #endif
57     if (!cmd.empty()) {
58         std::system(cmd.c_str());
59     }

```

Finally we obtain the converted image output:

